

DeepSeek V4 万字拆解：逼近闭源天花板，价格却只要 1/6

原创 老许漫谈Allnfra 老许漫谈Allnfra 2026年4月25日 14:51 上海

4 月 24 日这天，GPT-5.5 和 DeepSeek V4 同日发布。所有人都在聊 GPT-5.5，但真正让圈内人坐不住的，是 V4 的价格。

TechCrunch: "DeepSeek closes the gap with frontier models"

VentureBeat: "near state-of-the-art intelligence at 1/6th the cost"

Simon Willison: "almost on the frontier, a fraction of the price"

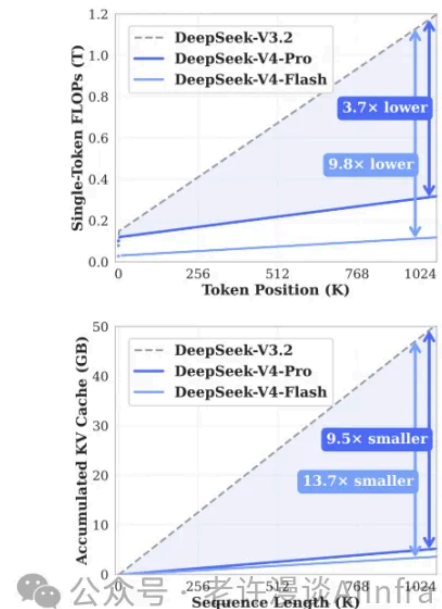
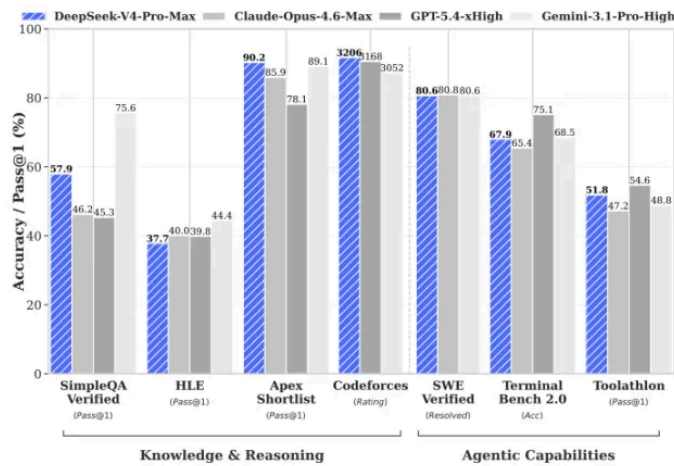
三句话，同一个意思：开源模型快追上闭源了，而且便宜得多。

本文导读：全文约 12000 字，预计阅读 15 分钟。内容覆盖 DeepSeek V4 的四个核心维度——

- 第一章 架构革新：CSA + HCA 混合注意力、mHC 流形约束超连接
- 第二章 训练革新：Muon 优化器、FP4 量化感知训练、EP 通算融合
- 第三章 推理革新：KV Cache 结构革新 + SSD 分层存储
- 第四章 后训练：专家训练、在线策略蒸馏、RL 基础设施

开篇：开源最强，逼近闭源——凭什么这么便宜？

先看结果。DeepSeek-V4 在六大核心基准上全面登顶开源第一：



DeepSeek V4 核心基准测试结果

知识层面，SimpleQA-Verified 57.9%，领先第二名开源模型 20 个百分点以上；Chinese-SimpleQA 84.4%，开源最强。推理层面，HLE 37.7%，Apex 38.3%，Codeforces 3206 Rating——三项均为开源第一。编程层面，SWE-Verified 80.6%，LiveCodeBench 93.5%，与 GPT-5.4 基本持平。Agent 层面，TerminalBench 2.0 达到 67.9%，BrowseComp 83.4% 仅落后 GPT-5.5 一个百分点。长上下文方面，1M Token 场景下 MRCR 83.5%，甚至超越了 Gemini-3.1-Pro 的 76.3%。

两个型号的具体参数：

DeepSeek V4 Pro 和 Flash 模型参数对比

DeepSeek-V4-Pro 拥有 1.6T 总参数、49B 激活参数，当前最大的开源权重模型；DeepSeek-V4-Flash 拥有 284B 总参数、13B 激活参数，轻量高效。两者均支持 100 万 Token 上下文长度，在 32T+ Token 上预训练，MIT 协议开源。

但跑分只是上半场。真正的震撼在下半场——

算力效率：DeepSeek-V4-Pro 在百万 Token 场景下，单 Token 推理 FLOPs 仅为 V3.2 的 27%，KV Cache 仅为 10%。V4-Flash 更是做到 FLOPs 10.2%，KV Cache 7.3%。

价格：V4-Pro API 定价 \$1.74 / 百万输入 Token + \$3.48 / 百万输出 Token。同样一百万 Token 输入+输出，GPT-5.5 要 \$35，Claude Opus 4.7 要 \$30，DSV4-Pro 只要 **\$5.22**——大约是闭源顶流的 **1/6 到 1/7**。V4-Flash 更极端，只要 \$0.42，几乎是闭源的 **1/100**。

翻译成成人话：不是"更贵所以更强"，而是"更省却几乎一样强"。

这意味着什么？V4-Flash 百万 Token 只要 \$0.42——你可以把一整本书喂给模型做分析，成本不到一杯咖啡。以前只有大厂玩得起的长上下文推理，现在创业团队甚至个人开发者都能日常使用了。

DeepSeek 自己在论文里说：性能略落后 GPT-5.4 和 Gemini-3.1-Pro，"developmental trajectory that trails frontier models by approximately 3 to 6 months"。VentureBeat 的第三方评测也印证了这一点——GPQA Diamond 上 DSV4 90.1% vs GPT-5.5 93.6% vs Opus 4.7 94.2%，差距在个位数百分点。但 DSV4 的价格只有对方的几分之一。

这才是 DeepSeek V4 真正让人坐不住的地方——性能逼近天花板，价格却从天花板直接砍到地板。效率的质变意味着百万 Token 上下文从"偶尔用一下"变成了"日常可用"，开源大模型的部署门槛直接砍掉了一大截。

它是怎么做到的？

DeepSeek V4 的效率质变来自四个层面：

- **架构层面：**混合注意力（CSA + HCA）替换 MLA，流形约束超连接（mHC）升级残差连接
- **训练层面：**Muon 优化器替代 AdamW，FP4 量化感知训练，EP 通算深度融合
- **推理层面：**KV Cache 结构革新 + SSD 分层存储，突破 GPU 显存天花板
- **后训练层面：**专家训练 + 在线策略蒸馏（OPD），从"会做题"到"会做事"

下面我们逐层拆解，重点讲清楚"为什么要这么变"。

第一章：架构革新——从 MLA 到混合注意力 + mHC

1.1 为什么 V3 的 MLA 不够用了？

V3 已经很强了，但在超长上下文场景下有一个根本性问题：**注意力的计算成本随上下文长度二次增长**。

想象你在一间有一万人的会议室里开会——每个人都要听其他所有人的发言，这就是标准注意力的做法。当"会议室"扩展到百万 Token 时，计算量和显存消耗直接爆炸。

V3 用了 MLA (Multi-head Latent Attention, 多头潜在注意力) 来压缩 KV Cache，但压缩率到了百万 Token 仍然不够——显存撑不住，推理太慢。V4 的解法不是"更狠的压缩"，而是彻底换了一种注意力架构——从 MLA 演进到 CSA + HCA 混合注意力。要理解这个演进，我们需要从 Attention 本身开始，一步步讲清楚"为什么要这么变"。

1.2 标准注意力—— $O(n^2)$ 的代价

要理解 V4 做了什么，先得搞清楚 Attention 本质上在干什么。

标准注意力机制 QKV 架构

Attention 就像是"每个人翻阅所有人的笔记，找到和自己相关的部分"。具体来说，标准 Multi-Head Attention 的计算分三步：

第一步：投影—— $O(n \times d^2)$

输入是一个 $n \times d$ 的矩阵 H (n 个 Token，每个 Token 有 d 维隐藏状态)。三个投影矩阵分别把 H 投影成 Q 、 K 、 V ：

$Q = H \cdot W_Q$ ，维度 $n \times d_k$ (n 个 Token 各自的查询向量)

$K = H \cdot W_K$ ，维度 $n \times d_k$ (n 个 Token 各自的键向量)

$V = H \cdot W_V$ ，维度 $n \times d_v$ (n 个 Token 各自的值向量)

为什么需要投影成 Q 、 K 、 V 三份？因为它们扮演三个不同的角色：

- Q (Query)："我在找什么信息"——当前 Token 的需求
- K (Key)："我有什么信息可以被找到"——每个 Token 的标签/索引
- V (Value)："找到我之后，拿走的内容"——每个 Token 的实际信息载荷

如果不做投影，直接用 $H \cdot H^T$ 算注意力，那"找什么"和"被找到的标签"就是同一个表示，模型没有能力区分"我需要什么"和"我能提供什么"。投影就是让模型学会从同一个输入中提取出三种不同的语义角色。

打个比方：你去图书馆找书—— Q 是你脑子里的需求"我想看关于注意力机制的书"， K 是书架上的标签"机器学习""NLP""计算机视觉"， V 是书的实际内容。需求和标签必须是不同的表示空间，匹配上之后拿到的是内容。三者职责不同，所以需要三个独立的投影矩阵。

注意：这一步的复杂度是 $O(n \times d \times d_k)$ ，对 n 是线性的。投影本身不是瓶颈。

第二步：算注意力分数—— $O(n^2 \times d_k)$ ，瓶颈在这里

$$\text{Score} = Q \cdot K^T / \sqrt{d_k}$$

Q 是 $n \times d_k$ 的矩阵， K^T 是 $d_k \times n$ 的矩阵。两者相乘，得到一个 $n \times n$ 的注意力分数矩阵——矩阵的每个元素 (i, j) 表示第 i 个 Token 对第 j 个 Token 的关注程度。

这就是 $O(n^2)$ 的来源： n 个 Query，每个都要和 n 个 Key 算点积，总共 $n \times n$ 次计算。当 $n = 100$ 万时，这个矩阵有 10^{12} 个元素。

第三步：加权求和—— $O(n^2 \times d_v)$

$$\text{Output} = \text{Softmax}(\text{Score}) \cdot V$$

$\text{Softmax}(\text{Score})$ 是 $n \times n$ 的矩阵， V 是 $n \times d_v$ 的矩阵。相乘得到 $n \times d_v$ 的输出。这一步也是 $O(n^2)$ 的。

总计算复杂度： $O(n \times d^2) + O(n^2 \times d_k) + O(n^2 \times d_v) \approx O(n^2 \times d)$ （当 $n \gg d$ 时，第二步和第三步的 n^2 项占主导）。

KV Cache 的大小： $O(n \times n_h \times d_h)$

推理时是自回归生成——每次生成一个新 Token，都要和之前所有 Token 的 K 、 V 做注意力计算。所以必须把之前所有 Token 的 K 和 V 存下来，这就是 KV Cache。

对于 Multi-Head Attention，每个 Token 有 n_h 个注意力头，每个头的 K 和 V 各有 d_h 维：

$$\text{单层 KV Cache} = n \times n_h \times 2 \times d_h \quad (n \text{ 个 Token} \times n_h \text{ 个头} \times K \text{ 和 } V \text{ 各 } d_h \text{ 维})$$

所以 KV Cache 的大小正比于序列长度 n 。当 $n = 100$ 万、 $n_h = 128$ 、 $d_h = 128$ 时，单层 KV Cache 就需要约 12.5 GB（BF16 精度）。一个几十层的模型，KV Cache 轻松超过 100 GB——GPU 显存根本放不下。

小结： $O(n^2)$ 的计算瓶颈来自第二步 $Q \cdot K^T$ ； $O(n)$ 的 KV Cache 瓶颈来自需要存储所有历史 Token 的 K 和 V 。两个瓶颈的根源都是序列长度 n ——后续 MLA、CSA、HCA 的所有改进，都

是围绕"怎么让 n 不再是瓶颈"展开的。

1.3 MLA——压缩了特征维度，但没压缩序列长度

DeepSeek V2/V3 引入的 MLA（Multi-head Latent Attention）是第一次关键突破。核心思想：**不直接存 KV，而是存一个低维的潜在向量。**

MLA 低秩 KV 压缩架构

MLA 的关键操作是**低秩分解**。原先每个 Token 要存 n_h 组 K 和 V（每组 d_h 维），总共 $n_h \times d_h$ 维。MLA 把这个大矩阵分解成两个小矩阵的乘积：

KV 压缩（下投影）： $c_{KV} = h \cdot W_{DKV}$ （从 d 维压到 d_c 维， $d_c \ll n_h \times d_h$ ）

KV 恢复（上投影）： $K = c_{KV} \cdot W_{UK}$ ， $V = c_{KV} \cdot W_{UV}$ （从 d_c 维恢复到 $n_h \times d_h$ 维）

推理时，Cache 里只存压缩后的 c_{KV} （ d_c 维），用到 K 和 V 的时候再实时恢复。这样 KV Cache 从 $O(n \cdot n_h \cdot d_h)$ 降到了 $O(n \cdot d_c)$ ，压缩了约 8–10 倍。

类比：MLA 就像是“不存整本书，只存目录”——需要查原文时再从目录定位。目录比书薄得多，但查书需要额外一步。

MLA 的瓶颈：序列长度 n 还是在那里。它压缩了每个 Token 的 KV 维度，但没有压缩 Token 的数量。当 $n = 1M$ 时，即使每个 Token 只存 512 字节，KV Cache 仍然需要 512MB——加上 Attention 计算本身的 $O(n^2)$ FLOPs，MLA 的压缩已经到头了。

为什么 MLA 无法继续？ 因为 MLA 的压缩是在“特征维度”上做的——把每个 Token 的 KV 从高维压到低维。但 $O(n^2)$ 复杂度的根源不是特征维度，而是**序列长度**。不在序列维度上动刀，再怎么压缩特征维度也无济于事。这就是 V4 的切入点。

1.4 CSA——不仅压缩维度，还压缩序列长度

CSA（Compressed Sparse Attention）在 MLA 的基础上走了关键的一步：**不仅压缩每个 Token 的 KV 向量维度，还压缩序列本身。**

先看 CSA 的整体架构图：

CSA 架构图：Token 级压缩 + Lightning Indexer 稀疏选择

上图清晰展示了 CSA 的三个核心模块：左边的 Token-Level Compressor 把连续 m 个 Token 压缩成一个 KV 条目；中间的 Lightning Indexer 做稀疏选择，只挑出最相关的 k 个压缩块；底部的 Sliding Window KV Entries 补充局部精细依赖。

模块一：Token 级压缩器——把序列砍短

CSA 模块一：Token级压缩器——重叠压缩机制

CSA 首先计算两组 KV 条目及其对应的压缩权重：

$$C_a = H \cdot W_{aKV}, C_b = H \cdot W_{bKV} \text{ (两组 KV 条目)}$$

$$Z_a = H \cdot W_{aZ}, Z_b = H \cdot W_{bZ} \text{ (对应的压缩权重)}$$

为什么是两组？因为 CSA 使用了**重叠压缩策略**。第 i 个压缩块不仅包含第 $[m_i, m(i+1))$ 段的 a 组条目，还包含第 $[m(i-1), m_i)$ 段的 b 组条目：

$$[S_a; S_b] = \text{Softmax_row}([Z_a + B_a; Z_b + B_b])$$

$$C_i^{\text{Comp}} = \Sigma(S_{a_j} \odot C_{a_j}) + \Sigma(S_{b_j} \odot C_{b_j})$$

这里的 B_a 、 B_b 是可学习的位置偏置。重叠压缩的设计非常巧妙——就像裁缝缝紉时布料的搭接，确保相邻压缩块的边界信息不丢失。如果只用一组做无重叠压缩，块与块之间的“缝隙”会丢掉关键信息。

V4-Pro 中 $m = 4$ ，意味着每 4 个 Token 压缩成 1 个，序列长度直接缩短到 1/4。

模块二：Lightning Indexer 稀疏选择——只精读关键段落

CSA 模块二：Lightning Indexer 稀疏选择机制

压缩完序列长度之后，CSA 并没有对所有的压缩块做全量 Attention——那样仍然太贵。它引入了一个“闪电索引器”，就像搜索引擎的倒排索引一样，先快速定位“哪几页可能相关”，再精读那几页。

具体计算过程：

生成查询向量： $c_Q = h_t \cdot W_{DQ}$ （下投影到 d_c 维）

生成索引查询： $q_I = c_Q \cdot W_{IUQ}$ （上投影到索引头维度 c_I ）

同样对压缩块生成索引键： K^IComp （与主 KV 共享同一套压缩流程）

计算索引得分： $l(t,s) = \sum_h w_h^l \cdot \text{ReLU}(q_{\{t,h\}}^l \cdot K_s^l \text{Comp})$

Top-k 选择：只保留得分最高的 $k = 512$ 个压缩 KV 块

这里有几个值得注意的细节：(1) 索引查询走的是独立的投影路径 ($W_{DQ} \rightarrow W_{IUQ}$)，与主 Attention 的查询分离，避免互相干扰；(2) 用 ReLU 而非 Softmax 做非线性激活，因为索引只需要"谁更相关"的排序信号，不需要概率分布；(3) 每个索引头有独立的权重 w_h^l ，可学习，让不同头关注不同类型的匹配模式。

模块三：Shared KV MQA + Grouped Output Projection

CSA 模块三：Shared KV MQA + 分组输出投影

选出的 512 个压缩 KV 条目，同时充当 Key 和 Value (Multi-Query Attention) ——所有注意力头共享同一组 KV，进一步压缩存储和计算量。

最终输出经过 Grouped Output Projection：把 n_h 个头的输出分成 g 组，每组先投影到中间维度 d_g ($d_g < c \cdot n_h / g$)，再合并投影回原始维度 d 。这种"先压缩再合并"的策略，与 MLA 的低秩压缩一脉相承。

1.5 HCA——极端压缩，"扫一眼标题就行"

CSA 通过稀疏选择保留了 512 个压缩块做精确 Attention，但这意味着它只能"精读"关键段落，对全局的"模糊印象"不够。HCA（Heavily Compressed Attention）来补这个缺——把每 $m' = 128$ 个 Token 压缩成 1 个，不做稀疏选择，全量 Attention。

先看 HCA 的架构图：

HCA 架构图：重度压缩 + 全量 Attention

可以看到，HCA 的结构与 CSA 相似，但关键区别是：Token-Level Compressor 使用了更大的压缩率（128:1 vs 4:1），且没有 Lightning Indexer——压缩完直接做全量 Attention。

HCA 的压缩公式比 CSA 更简洁（因为不需要重叠）：

$$C = H \cdot W_{KV}, Z = H \cdot W_Z$$

$$S = \text{Softmax_row}(Z + B)$$

$$C_i^{\text{Comp}} = \sum S_j \odot C_j \text{ (无重叠压缩, } m' = 128)$$

为什么不需要重叠？因为 HCA 的定位是"扫一眼标题"，它不需要块与块之间的精确边界信息——128 个 Token 压缩成 1 个，本来就是在保留"大致印象"，而不是精确定位。重叠压缩的设计是为了精确定位，那是 CSA 的职责。

同样，HCA 也使用 Shared KV MQA 和 Grouped Output Projection，与 CSA 的策略一致。

1.6 CSA + HCA 为什么必须搭配用？

用一句话概括：CSA 负责精读，HCA 负责泛读。

	CSA	HCA
压缩比	$m = 4$ （轻压缩）	$m' = 128$ （重压缩）
选择策略	Lightning Indexer Top- $k = 512$	全量 Attention
保留信息	精确——"精读关键段落"	模糊——"扫一眼标题"
适合场景	需要精确匹配的事实检索	长距离语义依赖
复杂度	$O(n \cdot k / m)$	$O((n/m')^2)$

单独用 CSA 的问题是：只选了 k 个块，长距离依赖可能丢失。单独用 HCA 的问题是：压缩太狠，精确匹配能力不足。

V4 的方案是**两者交错配置**——在网络的不同层交替使用 CSA 和 HCA：CSA 层精读关键信息，HCA 层捕获全局语义，互补而不冲突。V4-Pro 的前两层使用纯滑动窗口注意力，后续层 CSA 和 HCA 交错排列。

还有三个辅助机制

除了 CSA 和 HCA 的核心架构，V4 还引入了三个辅助机制来保证混合注意力的质量：

Sliding Window Attention：在 CSA 和 HCA 中，每个 Query 只关注前面的压缩块，无法访问同一压缩块内的其他 Token（因果性限制）。同时，最近的 Token 通常与当前 Token 最相关。所以 V4 额外引入了一个滑动窗口分支——为每个 Query 保留最近 n_{win} 个未压缩的 KV 条目，与压缩 KV 条目一起参与 Attention 计算。

Partial RoPE：CSA 和 HCA 的压缩 KV 条目同时充当 Key 和 Value，导致核心 Attention 输出会携带绝对位置信息（来自 KV 条目的加权求和）。V4 的解法：对每个 Query 和 KV 向量的最后 64 维应用 RoPE，并在 Attention 输出上对最后 64 维应用位置为 $-i$ 的 RoPE，抵消绝对位置编码，让输出携带相对位置信息。

Attention Sink：在 CSA 和 HCA 的核心 Attention 中，设置了一组可学习的 sink logits $\{z'_1, z'_2, ..., z'_{\{n_h\}}\}$ 。对第 h 个注意力头， $\text{Exp}(z'_h)$ 被加到 Softmax 的分母中，作为"注意力吸收

池"——防止某些异常 Token 吸收过多注意力分数，导致其他 Token 的注意力权重趋近于零。

效果有多猛？

在百万 Token 上下文下，单 Token 推理 FLOPs 从 V3.2 基线降到 V4-Pro 的 27%、V4-Flash 的 10.2%；KV Cache 大小从基线降到 V4-Pro 的 10%、V4-Flash 的 7.3%。

1.7 mHC——给残差连接装上"护栏"

残差连接是 Transformer 里信息从底层传到顶层的通道。V4 把它从"单车道"升级成了"多车道 + 护栏"，经历了三次演进。

mHC 残差连接演进：标准残差 $\rightarrow$ HC $\rightarrow$ mHC

1.7.1 标准残差——单车道，容易堵

$$x_{\text{out}} = x + F(x)$$

所有信息挤在一条 d 维通道里。 $F(x)$ 太大则信号被淹没， $F(x)$ 太小则梯度消失——跷跷板效应，两头压不住。

1.7.2 HC——多车道，但没红绿灯

$$X_{\{l+1\}} = B_l \cdot X_l + C_l \cdot F_l(A_l \cdot X_l)$$

HC 把残差流扩展到 $n_{\text{hc}} \times d$ 维（V4 中 $n_{\text{hc}} = 4$ ），引入三个可学习映射： A_l 选择哪些通道送进当前层， B_l 控制残差流在通道间如何混合， C_l 决定当前层输出怎么混入残差流。

核心优势：网络能动态调整不同深度特征的连接强度，甚至重排层的顺序。

核心问题： B_l 是自由矩阵，没有约束。多层堆叠时信号可能在某些通道上放大（梯度爆炸），也可能正负抵消（训练崩掉）。HC 论文自己承认"severe training instability"。

1.7.3 mHC——多车道 + 护栏 + 限速

mHC 保留 HC 的多车道和动态路由，但给 B_l 加上约束：把它限制在双随机矩阵流形上——每一行之和等于 1、每一列之和等于 1、所有元素非负。

$$B_l \in M = \{\text{矩阵 } M \mid M \cdot \mathbf{1} = \mathbf{1}, \mathbf{1}^T \cdot M = \mathbf{1}^T, M \geq 0\}$$

上图展示了每个 Transformer Block 中 mHC 的 Residual Mixing 结构：输入信号通过 A_I 选择通道后进入层变换 F_I，输出通过 C_I 混入残差流，同时 B_I 在残差流的 n_hc 个通道之间做可控混合。

问题：为什么双随机约束能解决 HC 的训练不稳定？

三个关键性质：

- 1. 谱范数 ≤ 1 ——信号经过 B_I 混合后只会变小或不变，不会放大，梯度不会爆炸
- 2. 对乘法封闭——多层堆叠后整体变换仍在流形上，不会"前几层还好，堆多了就崩"
- 3. 恢复了恒等映射—— $B_I = I$ 时退化为标准残差，最差也不会比标准残差差

	标准残差	HC	mHC
通道	1 条	n_hc 条	n_hc 条
路由	无	动态	动态
速度限制	无	无	谱范数 ≤ 1
护栏	无	无	非负 + 行列和 = 1
恒等映射	✓	×	✓
训练稳定性	好（但弱）	差	好

mHC 效果更好，是因为同时做到了三件之前互相矛盾的事：多通道动态路由、流形约束保稳定、恢复恒等映射保底线。标准残差只能做到后两件，HC 只能做到第一件，mHC 是第一次三者兼得。

工程上，V4 团队通过融合 Kernel 和重计算策略，把 mHC 的 wall-time 开销压到只有 6.7%。

第二章：训练革新——Muon + FP4 + EP 通算融合

本章聚焦训练层面的关键变化。Muon 优化器是核心亮点，单列一节展开；FP4 量化、TileLang 框架、EP 融合和长上下文并行合并介绍。每个话题展开都需要很大篇幅，这里先把关键点讲清楚，后续逐个深入。

2.1 Muon 优化器——为什么不用 AdamW 了？

AdamW vs Muon 优化器架构对比

AdamW 过去是大模型训练的标配，但它有个根本性的设计局限：逐元素更新，忽略矩阵整体结构。

AdamW 的更新规则是对每个参数元素独立计算一阶动量 M 和二阶动量 V ，然后用 $M/(\sqrt{V} + \epsilon)$ 做自适应缩放。问题在于：如果你对 AdamW 的更新矩阵做 SVD 分解，会发现不同方向的奇异值不相等——主方向的更新幅度远大于其他方向，模型会偏向沿着少数主方向更新，忽略了其他可能有价值的探索方向。

Muon 的核心思想完全不同：抛弃逐元素缩放，对整个更新矩阵做正交化。具体流程：

- Step 1: 计算动量 $M = \beta \cdot M + (1-\beta) \cdot G$ （和 AdamW 一样）
- Step 2: SVD 分解 $M = U\Sigma V^T$ ，把动量矩阵分解为正交矩阵 U 、 V 和对角矩阵 Σ
- Step 3: Newton-Schulz 迭代（5 次）近似求解矩阵符号函数 $\text{sign}(M) = UV^T$ ——把所有奇异值变成 1，只保留更新方向，丢弃更新幅度
- Step 4: 更新权重 $W = W - \eta \cdot \sqrt{(\max(A,B))} \cdot 0.2 \cdot UV^T - \lambda W$

为什么"只保留方向、丢弃幅度"反而更好？因为 AdamW 的自适应缩放会让模型过度依赖少数主方向，限制了探索空间。Muon 强制所有方向的更新幅度相等，让模型在训练中"探索更多方向"——不是只走最快的路，而是同时试探多条路。

但 Muon 不能直接替换 AdamW。大模型里有些参数是矩阵（如线性层权重），有些是标量（如 RMSNorm 缩放因子、嵌入层）。Muon 只能处理矩阵参数，标量参数仍然用 AdamW。所以 V4 采用混合优化策略：

参数类型	优化器	原因
矩阵参数（线性层权重等）	Muon	矩阵正交化有意义
标量参数（RMSNorm、Embedding）	AdamW	标量无法做矩阵正交化

为了让两种优化器的更新幅度对齐，V4 引入了固定缩放因子 0.2 和 $\sqrt{\max(A,B)}$ 的 RMS 修正——确保不同形状的矩阵参数更新幅度一致，且与 AdamW 的典型更新幅度匹配。

分布式 Muon：ZeRO-1 会对优化器状态分片，但 Newton-Schulz 正交化需要完整的动量矩阵。V4 的方案：平时分片存储，只在正交化时 All-Gather 拼回完整矩阵，正交化后再只保留本地分片。额外通信开销仅 1%–3%，但优化器内存只有 AdamW 的一半（Muon 只需一阶动量，不需要二阶动量）。

2.2 训练基础设施四件套

训练基础设施四件重武器

除了 Muon，V4 在训练层面还有三个关键改进，和 Muon 一起构成"训练四件套"：

FP4 量化感知训练——V4 在预训练阶段就引入了 FP4 量化，工业级大模型的首例。以前 FP4 量化只在推理时做，训练用 FP8 或 BF16。V4 直接在训练中用 4-bit 权重，配合梯度补偿机制，精度几乎无损，权重存储直接砍半。当前硬件上 FP4×FP8 的峰值算力与 FP8×FP8 相同，但未来硬件理论上可以再快 1/3。

TileLang 声明式 Kernel 框架——手写 CUDA Kernel 费时费力，TileLang 让你描述"要算什么"，它自动生成优化后的 CUDA 代码。这让 V4 团队能快速迭代各种定制算子，缩短开发周期。

EP 通算深度融合 + 长上下文并行——MoE 的专家并行（EP）通信开销很大，V4 把通信和计算在 Kernel 级别融合，通信线程搬数据的同时计算线程在做矩阵乘，GPU 不再"等通信"。百万 Token 的注意力计算远超单卡显存，Contextual Parallelism 把长序列切分到多卡，配合扩展自动微分实现灵活的激活检查点，显存峰值压到可控。

第三章：推理革新——让百万 Token 日常可用

3.1 KV Cache 革命

GPU + SSD 分层 KV Cache 存储

V4 针对 CSA/HCA 混合注意力设计了专门的 KV Cache 结构，打破了 PagedAttention 的基本假设——不同层的 Cache 策略不同（CSA 压缩块 vs HCA 压缩块 vs Sliding Window 原始 Token），无法统一管理。

V4 的方案是双层 Cache：经典 KV Cache 存放 CSA/HCA 的压缩 KV 条目；State Cache 存放 SWA 的滑动窗口 KV 和尚未压缩的尾部 Token。

CSA 层的 KV Cache 按稀疏模式存储，只保留关键 Token；HCA 层的 KV Cache 用极端压缩，维度极低。两者协同，使 KV Cache 总量降到 V3.2 的 10%。混合精度存储方面，RoPE 维度用 BF16，其余用 FP8，相比纯 BF16 再砍一半。

3.2 SSD 卸载——突破 GPU 显存天花板

百万 Token 的 KV Cache 再怎么压缩，也可能超出 GPU 显存。V4 的方案：**GPU 显存不够，用 CPU 侧的 SSD 来凑。**

核心思路很简单——活跃数据放 GPU 显存（快但贵），冷数据放 CPU 侧 SSD（大但慢），需要时通过 PCIe 总线用 H2D/D2H 换入换出。就像电脑的内存和硬盘，谁也不指望把所有数据都放内存里。

SSD 分层 KV Cache 存储架构

具体来说，两种 KV 条目的处理策略完全不同：

CSA/HCA 压缩 KV 条目——体积小，全量存 SSD。已极度压缩（CSA 4:1，HCA 128:1），直接全部写入 CPU 侧 SSD。当新请求命中共享前缀时，从 SSD 读取压缩 KV，经 PCIe 走 D2H → H2D 路径加载回 GPU 显存复用，跳过重复 Prefill。尾部不完整压缩块仍需重计算。

SWA 滑动窗口 KV 条目——体积大（约压缩 KV 的 8 倍），不能全量存。V4 设计了三种策略：

策略	存什么	恢复时怎么算	适用场景
全量缓存	所有 Token 的 SWA KV	读最后 n_{win} 个 Token 即可	SSD 充足、追求零冗余
周期检查点	每 p 个 Token 存一次快照	加载最近检查点 + 重算尾部	中间地带， p 值可调
零缓存	不存任何 SWA KV	重算 $n_{win} \times L$ 个 Token（利用压缩 KV）	SSD 紧张、算力充足

一句话总结：GPU 存热数据，SSD 存冷数据，三种策略按需选——这是百万 Token 上下文推理不再受限于 GPU 显存大小的关键。

第四章：后训练——从"会做题"到"会做事"

4.1 专家训练：专项特训

预训练让模型获得广博知识，但特定能力（编程、数学、Agent）需要专项特训。V4 对每个能力域设计了专门的训练数据和策略——先 SFT 建立基础能力，再用 GRPO 强化学习进一步优化。每个领域得到一个专家模型。

4.2 在线策略蒸馏（OPD）：大模型教小模型

多个专家模型需要合并成一个统一模型。OPD 让统一模型作为学生，优化与教师模型的 Reverse KL 散度——不是简单记忆答案，而是学习输出分布。V4-Flash 也通过 OPD 从 V4-Pro "偷师"，保证了小模型在推理、编程等任务上的竞争力。

4.3 RL 基础设施：百万 Token 场景的强化学习

RL 训练需要大量 Rollout（模型自己生成回答），百万 Token 上下文的 Rollout 极其昂贵。V4 团队设计了两个关键系统来解决这个问题。

可抢占容错的 Rollout 服务

为什么会发生抢占？ 大规模 GPU 集群里，GPU 是共享资源。集群有一个全局的抢占式调度器——任何正在运行的任务都可能被更高优先级的任务抢占。举个例子：你正在用 8 张卡跑 RL Rollout，这时一个更高优先级的推理服务需要扩容，调度器会直接把你的 8 张卡拿走，你的 Rollout 任务立刻中断。

在百万 Token 场景下，一次 Rollout 可能生成了几十分钟、消耗了大量 KV Cache，被抢占一次就意味着全部作废——如果从头重新生成，不仅浪费算力，还会引入"长度偏见"（较短的回答更容易在抢占中存活，导致模型偏向产生短序列）。

V4 的解法是 **Token 粒度 WAL + KV Cache 断点保存**，让 Rollout 像视频播放一样可以"暂停/恢复"：

可抢占容错 Rollout 服务架构

1. **运行时**: 模型每生成一个 Token，立刻追加写入该请求的 WAL 日志
2. **被抢占时**: 暂停推理引擎，把未完成请求的 KV Cache 保存下来
3. **恢复时**: 用 WAL 中已生成的 Token + 保存的 KV Cache，从断点继续解码
4. **硬件故障时**: 即使 KV Cache 丢失，也可以用 WAL 中持久化的 Token 重跑 Prefill 重建

与 Async RL 的关系：可抢占容错是 Async RL 的基础设施支撑。Async RL 的核心思想是 Rollout 生成和梯度更新**异步执行**——Rollout 跑在可能被抢占的 GPU 上，梯度更新跑在稳定的训练 GPU 上。没有可抢占容错，抢占意味着数据丢失和训练中断；有了 WAL + KV Cache 保存，Rollout 可以随时被抢占、随时恢复，训练过程不中断。

Agentic AI 沙箱：DSec

Agent 训练需要让模型在真实环境中执行操作（写代码、跑测试、浏览网页），这些操作必须隔离、安全、可复现。DSec 就是为此打造的生产级沙箱平台，单个集群可管理数十万并发实例。

DSec 沙箱平台架构

DSec 的核心设计围绕四个维度：

统一接口 + 四级隔离：四种执行底座（Function Call / Container / microVM / fullVM）共享同一套 API，切换只需改一个参数，从轻量函数调用到完整操作系统全覆盖。

分层存储加速启动：基于 3FS 分布式文件系统，容器用 EROFS 按需加载，microVM 用 overlaybd 共享只读层 + 本地 CoW 写入层，镜像秒级就绪。

密度优化：虚拟化环境去重页缓存 + 内存超卖回收 + 容器运行时自旋锁优化，单机可承载数千并发沙箱。

轨迹日志 + 抢占安全恢复：每条命令及结果持久记录，训练被抢占后回放缓存结果加速恢复，同时避免非幂等操作重复执行出错。

总结：V4 到底变了什么？

DeepSeek V3 vs V4 效率对比

维度	DeepSeek-V3	DeepSeek-V4
注意力	MLA	CSA + HCA 混合注意力
残差连接	标准残差	流形约束超连接（mHC）
优化器	AdamW	Muon
权重精度	FP8 训练	FP4 量化感知训练
上下文	128K	1M（百万 Token）
百万 Token FLOPs	基线	仅 27%
百万 Token KV Cache	基线	仅 10%
EP 通信	基础重叠	Kernel 级深度融合
Kernel 开发	手写 CUDA	TileLang 框架化
KV Cache 存储	全量 GPU 显存	GPU + SSD 分层

开源追闭源，不再是"会不会"的问题，而是"什么时候"。DeepSeek V4 告诉我们：这一天，可能比所有人想的都近。

参考资料

[DeepSeek-V4 论文](https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf)

[DeepSeek-V4 模型权重（MIT 协议）](<https://huggingface.co/collections/deepseek-ai/deepseek-v4>)

[DeepSeek-V3 论文](<https://arxiv.org/abs/2412.19437>)

[TechCrunch: DeepSeek previews new AI model that 'closes the gap' with frontier models](<https://techcrunch.com/2026/04/24/deepseek-previews-new-ai-model-that-closes-the-gap-with-frontier-models/>)

[VentureBeat: DeepSeek-V4 arrives with near state-of-the-art intelligence at 1/6th the cost](<https://venturebeat.com/technology/deepseek-v4-arrives-with-near-state-of-the-art-intelligence-at-1-6th-the-cost-of-opus-4-7-gpt-5-5>)

[Simon Willison: DeepSeek V4—almost on the frontier, a fraction of the price](<https://simonwillison.net/2026/apr/24/deepseek-v4/>)

喜欢作者